Richiami di Java Script

Rev 2.2 del 01/09/2026

Richiami sui principali metodi js di accesso al DOM …….. 2
Richiami sui principali pseudoselettori CSS …….. 4
Form e Controlli : lettura e scrittura del value …….. 6

Evoluzioni di javascript …….. 7
Arrow Functions e Binding del this …….. 8
javascript funzionale …….. 10
typeof e instanceof …….. 15
Animate e FadeIn …….. 15

Delegated Events …….. 16

Le classi in ES5 e ES6 …….. 18
Distribuzione delle librerie …….. 23

Salvataggio di un testo nella clipboard …….. 25
Approfondimenti ……..26

Richiami sui principali metodi di accesso al DOM

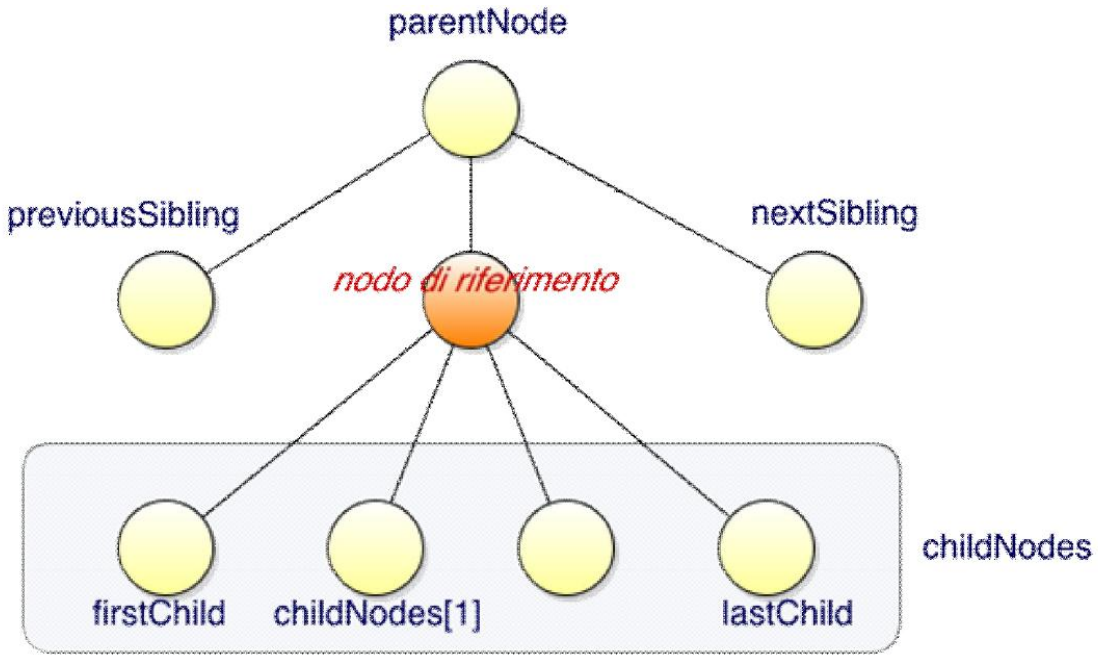
`document.`**`getElementById`** restituisce il singolo elemento, applicabile SOLO a document
`document.`**`getElementsByName`** restituisce un vettore di elementi, applicabile SOLO a document

`wrapper.`**`getElementsByTagName`** restituisce un vettore di elementi
`wrapper.`**`getElementsByClassName`** restituisce un vettore di elementi-
Applicabile anche su più classi contemporaneamente `getElementsByClassName ("class1 class2")`

`wrapper.`**`querySelector`**`("input[type=text]")` restituisce il primo input type=text
`wrapper.`**`querySelectorAll`**`("input[type=text]")` restituisce un vettore di input type=text
Il primo se non trova nulla restituisce undefined, il secondo se non trova nulla restituisce un vettore vuoto.

Molto comodi perchè, a differenza dei precedenti, accettano come parametro un qualunque selettore / pseudoselettore CSS. **Però molto più pesanti. Da usare una volta sola all'onload**

Struttura dell'albero HTML
Nell'immagine sotto rappresentata si ipotizza un puntatore attualmente posizionato sul "nodo di riferimento" visualizzato in rosso



`current.`**`children`** restituisce un vettore con tutti i tag **figli diretti** di current
Accetta come parametro SOLO un indice numerico : **`myFirstDiv = myWrapper.children[0]`**
`current.`**`childNodes`** come sopra però contempla anche i text nodes ed i comment nodes
`current.`**`children[i]`** i-esimo **figlio diretto**
`current.`**`firstElementChild`** primo **figlio diretto**
`current.`**`lastElementChild`** ultimo **figlio diretto**
`current.`**`children.length`** numero di **figli diretti**
`current.`**`firstChild`** come sopra però contempla anche i text nodes ed i comment nodes
`current.`**`lastChild`** come sopra però contempla anche i text nodes ed i comment nodes
`current.`**`nextElementSibling`** fratello successivo
`current.`**`previousElementSibling`** fratello precedente
`current.`**`next/previousSibling`** come sopra però contempla anche i text nodes ed i comment nodes
`current.`**`hasChildNodes()`** indica se il nodo corrente ha nodi figli
`current.`**`parentElement`** genitore di current
`current.`**`closest("div")`** ricerca il primo predecessore di tipo DIV (antenato) di current

`current.contains(span)` verifica se il tag puntato da **span** è un **descendant** di current, cioè se *span* coincide con *current* oppure se è un suo figlio o nipote.

`current.textContent` contenuto testuale del nodo (foglia)
`current.innerHTML` intero contenuto HTML

`current.cloneNode(true)` clona current. True fa si che vangano clonati anche tutti i descendant
Attenzione che **nel clone vengono 'persi' eventuali handler di evento**

`current.remove()` rimuove current dal DOM. L'elemento però rimane allocato in memoria e **continua a rimanere accessibile da codice** *(per rimuoverlo delete)*

`current.removeChild(current.firstElementChild)`

`document.createElement("div")` crea un nuovo HTMLElement . **Disponibile SOLO su document**
`current.appendChild(newChild)` appende child in coda a current. **Lo taglia dalla posizione corrente**
Non accetta come parametro una stringa HTML, nel qual caso occorre utilizzare .innerHTML
`current.append(node1, node2, )` consente di appendere più figli in coda a current
`current.prepend(node1, node2,)` consente di appendere più figli davanti al primo nodo di current

`current.insertBefore(newChild, current.children[0])` appende newChild come 1° figlio di current
`current.insertAdjacentElement(pos, newElement)` aggiunge un nuovo fratello/figlio
Il primo parametro **pos** può assumere i seguenti valori:

```
'beforebegin': Before the targetElement itself (fratello precedente)
'afterbegin': Just inside the targetElement, before its first child (primo figlio)
'beforeend': Just inside the targetElement, after its last child (ultimo figlio)
'afterend': After the targetElement itself (fratello successivo)
```

`current.matches("p, div")` verifica se current corrisponde ad un dei selettori indicati.
Notare che i selettori vengono indicati come stringa. Restituisce true / false
`current.tagName` legge il nome del tag (es P)
`current.nodeName` come sopra ma vale anche per gli attributi
`current.nodeType` tipo di nodo sulla base della seguente tabella:

```
1 = ELEMENT_NODE = nodo vero e proprio
2 = ATTRIBUTE_NODE = attributo
3 = TEXT_NODE = nodo testuale, cioè foglia dell'albero
5 = WHITESPACE_NODE =    a capo; etc.
8 = COMMENT_NODE = commento.  9 = DOCUMENT_NODE = l'intero documento
```

Cast `let present = Boolean(value)`

NodeList e htmlCollection

querySelectorAll restituisce una collezione di tipo **NodeList**
getElementsByName/TagName/ClassName restituiscono una **htmlCollection**
Sono entrambi oggetti iterabili simili ad un Array ma **NON** supportano i metodi di javascript funzionale e nemmeno lo splice(). L'unica differenza è che la **NodeList** supporta il forEach mentre **htmlCollection** no

Per convertire **NodeList** e **htmlCollecion** in **Array** :

```
let vet = Array.from(NodeList)      // oppure
let vet = [...NodeList]
```

Metodi per la gestione degli attributi

Attributi statici html

```
current.getAttribute(name);
current.setAttribute(name, value);
current.hasAttributes( );
current.hasAttribute(attributeName);
current.removetAttribute(name);
```

Attributi dinamici javascript : si utilizza semplicemente l'operatore **puntino**.

Selettori di attributo

I **selettori di attributo** permettono di individuare qualunque elemento del DOM sulla base dei suoi attributi HTML. Attenzione che all'interno dei selettori di attributo si possono usare **solo sugli attributi HTML statici cioè quelli definiti all'interno del file html oppure creati da javascript mediante .setAttribute().**
Non considerano invece gli **attributi dinamici assegnati tramite il puntino**.

Nota: tutti i selettori CSS lavorano sempre sul DOM javascript, quindi vedono sia gli elementi presenti nel file HTML, sia quelli creati dinamicamente tramite JavaScript (ma solo con setAttribute).
Esempi:

```
input[type=text]
input[type="radio"]
input[name=txtNome]
input[name="D'Alessio"]
```

Attenzione a **NON lasciare spazi** davanti alle quadre
Le **virgolette** sono facoltative.
Devono essere usate se il **value** dell'attributo

- contiene spazi o apici singoli
- non è stringa ma è un number

Attributi booleani

Per agli attributi booleani (ad esempio **checked**) esistono due diverse sintassi:
[checked] che riflette l'attributo statico checked
:checked che è una pseudoclasse che riflette la corrispondente proprietà dinamica.

Ovviamente le psudoclassi sono preferibili, però ne esistono ***solo*** 5 in tutto :

```
:checked
:disabled
:enabled      (uguale opposto a disabled)
:required     (campi obbligatori)
:read-only
:selected     (solo jQuery !! in javascript è mappato come :checked che si applica anche ai listBox)
```

Nel caso dell'attributo ***multiple*** di un listBox, non esiste la pseudo classe ma poiché l'attributo viene praticamente sempre settato a livello statico nel file html, si può tranquillamente usare ***[multiple]***

Attributi personalizzati

Possono essere creati tramite dataset : `div.dataset.userName='pippo'`

Un dataset non è semplicemente una variabile javascript. In corrispondenza di ogni dataset viene creato un attributo html statico visibile nell'inspector, utilizzabile all'interno di querySelector ed avente un prefisso **data-**. Cioè in pratica il camelCase viene tradotto in trattino.

```
let div = document.querySelector("div[data-user-name='pippo']")
```

Utilizzo combinato

I selettori di attributo possono anche essere applicati in forma combinata. Esempi:

```
querySelector("input[type=radio][name=opt1]:checked")
querySelectorAll("input[type=radio][data-user-name=4Bstudent]")
alert(querySelector('input[name="optGenere"]:checked').value)
```

Notare il seguente esempio: `querySelectorAll(":disabled")` restituisce tutti gli elementi attualmente disabilitati. In javascript per ri-abilitarli tutti occorre eseguire un ciclo.
Viceversa in jQuery sarebbe possibile applicare l'effetto all'intera collezione.

CSS Functions

```
:not(selettoreSecondario)    // Es input[type=radio]:not(:checked)
#menu li:has(ul)             // Gli elementi che contengono il descendant indicato
:contains(testo);            // solo jQuery. Restituisce quei tag che contengono il testo indicato
```

Pesudoclassi nth-child e nth-of-type

:first-child // Primo figlio **generico** (indipendentemente dal suo tipo)
:first-of-type // Primo figlio **del suo tipo**

:first-child restituisce true se l'elemento corrente **gode della proprietà di essere primo figlio (primogenito) del proprio genitore**, qualunque sia il genitore, e indipendentemente dal proprio tipo.

```
<div id="wrapper">
     <p> …… </p>
     <p> <input ……> </p>
     <input ……>
</div>
```

Il primo `<input>` presenta **:first-child** uguale a true perché è in assoluto il primo figlio di <p>
Il primo `<input>` presenta **:first-of-type** uguale a true perché, rispetto al proprio padre, è il primo input.
Il secondo `<input>` presenta **:first-child** uguale a false perché dentro wrapper, prima di lui, ci sono altri figli
Il secondo `<input>` presenta **:first-of-type** uguale a true perchè, all'interno di wrapper, è il primo input

:nth-child(i) // i-esimo figlio **generico** (indipendentemente dal tipo) (**a base 1**)
:nth-of-type(i) // i-esimo elemento **del suo tipo** (**a base 1**)

:last-child // Ultimo figlio **generico** (indipendentemente dal tipo)
:last-of-type // Ultimo elemento **del suo tipo**

#wrapper :nth-child(i) // i-esimo elemento **generico** contenuto in wrapper
#wrapper :nth-of-type(i) // i-esimo elemento **del suo tipo** contenuto in wrapper
:only-child // Figlio unico

Note

(1) Si consideri la seguente struttura:

```
<label>  <input type="radio" name="opt1" value="a">  </label>
<label>  <input type="radio" name="opt1" value="a">  </label>
```

```
input[type=radio][name=opt1]:nth-of-type(1)
```

non restituisce il primo radio button avente name=opt1, ma ***TUTTI*** i tag input che soddisfano alla proprietà di essere `:first-of-type` (ad esempio perché ognuno è il primo figlio di una label) e che hanno anche type=radio e name=opt1.

(2) Allo stesso modo

```
#wrapper p:first-of-type
```

non restituisce il primo tag `<p>` interno a #wrapper, ma tutti i tag `<p>` interni a #wrapper che soddisfano alla proprietà di essere `:first-of-type,` perché ad esempio ciascuno è inserito all'interno di una propria label.

(3) **:nth-child(i) e :nth-of-type(i)** non sono applicabili alle classi ma SOLO ai tag

even e odd

Gli pseudoselettori **:nth-child() e :nth-of-type()** oltre all'indice i-esimo accettano come parametro anche i valori: **even** tutti gli elementi pari - **odd** tutti gli elementi dispari.
Questi valori **sono utilizzabili soltanto all'interno di** **nth-child** e **nth-of-type.**
E' anche disponibile una firma del tipo **nth-of-type(3n - 2)**
prende tutti i multipli di 3 e sottrae 2, cioè: 1, 4, 7, 10, 13, etc

Puntualizzazioni su Form e Controlli : lettura e scrittura del value

in lettura

Nel caso di **select a selezione singola** restituisce automaticamente il **value della voce selezionata**.
Se il value della voce selezionata è vuoto restituisce il testo html della voce selezionata

Nel caso dei **radio button**, che sono esclusivi, tramite lo pseudo selettore ***:checked*** si può accedere all'unica voce selezionata e poi leggerne il value.
Per i radio non selezionati si può usare ***:not(:checked)*** e poi eseguire un ciclo.

Nel caso dei **check box**, si può accedere alle voci selezionate tramite lo pseudo selettore ***:checked***.
Dopo di che occorre necessariamente eseguire un ciclo. Per i non selezionati ***:not(:checked)***

Nel caso di **select a selezione multipla** la soluzione migliore è quella di scorrere le voci selezionate tramite un ciclo su **selectedOptions** :

```
for (let option of lst.selectedOptions)
```

oppure
```
let selectedOptions = lst.querySelectorAll("option:checked")
```
oppure
```
for (const option of lst.options)
  if(option.selected)
    msg += option.value + "\n";
```

In jQuery il metodo val() su un listbox multiple restituisce un vettore enumerativo con i value di tutte le voci selezionate, per i checkbox restituisce invece il value della PRIMA voce selezionata.

in scrittura

Nel caso del **ListBox a selezione singola** l'assegnazione del value provoca la selezione dell'elemento indicato. Se l'elemento non viene trovato il listbox verrà deselezionato (come selectedIndex=-1).
Nel caso del **ListBox a selezione singola** l'assegnazione del value provova la selezione dell'elemento indicato ma al contempo deseleziona tutti gli altri elementi, quindi non va bene. Si usa ***selectedOptions***

Nel caso di **radioButtons, checkBox** e **ListBox a selezione multipla** occorre necessariamente scorrere tutti gli elementi ed andare a settare opportunamente il campo **.checked** o **.selected**

```
for (const option of lst.options){
   option.selected = false
   if(option.value=="a" || option.value=="c")
      option.selected = true
}
```

*In jQuery è anche possibile passare al metodo **val()** un **vettore enumerativo di values** che, in tal caso, provoca la selezione di tutti gli elementi aventi il **value** indicato e la deselezione automatica delle altre voci*
```
$(':checkbox').val(['chk-1','chk-2']);
```
*Anche nel caso dei **radio button** (che sono esclusivi) occorre comunque 'passare' il value della voce da selezionare all'interno di un vettore enumerativo lungo 1. Se si passano più value viene selezionato l'ultim*

La gestione degli eventi

La selezione di un elemento da codice NON provoca MAI la generazione degli eventi, che dovranno eventualmente essere richiamati da codice

Esempio:
Dopo aver selezionato da codice una voce in un listbox occorre richiamare esplicitamente anche l'evento

```
list.addEventListener("change", esegui)
list.selectedIndex=2
esegui()
```

Evoluzioni di Java Script

1995 Netscape JavaScript
1996 **Standardizzazione ECMA** con il nome **ES** (**ECMA Script**), comunemente javascript
ECMA (European Computer Manufacturers Association) è un'associazione fondata nel 1961 e dedicata alla standardizzazione nel settore informatico e dei sistemi di comunicazione.
Dal 1994 viene chiamata ECMA International
1998 **ES2**

. . . . . .

2008 **Chrome V8 Engine** (scritto in C++)
2009 **ES5** (aggiunge fra le altre cose lo strict mode, le librerie JSON, i **web workers,** le promises)
2009 **Nodejs** per la programmazione lato server

2015 **ES6** o ES2015 – **programmazione funzionale**: iteratori, arrow functions, ES-Modules
2016 ES7 o ES2016 – assestamento dei cambiamenti introdotti nella precedente edizione.

Concetto di Function

Le **funzioni** sono oggetti (di tipo Function) **invocabili,** cioè a cui è possibile passare dei valori che non vengono semplicemente 'salvati' in memoria ma elaborati, e ritornano sempre un valore (eventualmente undefined). Come qualsiasi altro Object, le Functions possono essere assegnate ad una variabile e possono essere passate come parametro ad un'altra funzione.

Per definire una funzione esistono infatti due sintassi equivalenti:

```
function myFunction (name, surname){ }          // function declaration
let myFunction = function (name, surname){ }   // function expression
```

Le due sintassi sono perfettamente equivalenti. La seconda fa capire bene come la variabile myFunction contenga al suo interno l'indirizzo fisico della funzione all'interno della memoria.

Arrow Functions e Binding del this

Le Arrow Function, basate sull'operatore **lamda,** sono nate in ES6 **per**

- **semplificare sintatticamente la scrittura di una funzione**
- **migliorare la gestione del `this`.**

Sintassi Generale delle Arrow Function

La sintassi delle arrow function **omette** la parola chiave **function,** sostituita dall'operatore **lamda** ed i parametri vengono anteposti all'operatore lamda medesimo. La sintassi generale prevede infatti:

- le **parentesi tonde** intorno alla lista dei parametri da passare alla funzione di callback
- l'operatore **lamda**
- le **parentesi graffe** per delimitare il corpo della funzione

Forma tradizionale

```
let somma = function(x, y) {
     return x + y;
};
```

Arrow function

```
let somma = (x, y) => {return (x + y)}
```

L' **invocazione** di una arrow function è del tutto identica a quella di una normale funzione :

```
let totale = somma(3, 2);
```

Casi Particolari

- Se il corpo è costituito da una singola istruzione, si possono omettere ***sia* le parentesi graffe** per individuare il corpo della funzione ***sia* la scrittura diretta della parola return**. Ciò che viene ritornato dall'espressione interna (o dall'eventuale sottofunzione) viene automaticamente ritornato al chiamante. **Se però si mettono le parentesi graffe, allora il return diventa obbligatorio**

  ```
  (x, y) => x + y;
  (x, y) => {return x + y;}
  ```

- Se la funzione prevede un solo parametro, si possono omettere le parentesi tonde. esempio:

  ```
  x => x * 2
  ```

- Se la funzione non prevede parametri, l'uso delle parentesi tonde è obbligatorio :

  ```
  () => "Hello world!"
  ```

Utilizzo all'interno delle funzioni di callback

Grazie alla'estrema compattezza, le arrow function si prestano bene ad essere utilizzate nelle callback

Forma tradizionale

```
let numeri = [18, 13, 24];
numeri.forEach(function(valore) {
   console.log(valore);
});
```

Arrow function

```
numeri.forEach(valore => console.log(valore));  // console.log ritorna void
```

Il Binding del this

All'interno di una classe il ***this*** punta sempre alla classe stessa.

La peculiarità delle arrow function è che **non ridefiniscono il `this` che mantiene pertanto il significato che aveva all'interno dello scope genitore a monte della arrow function medesima**.
In particolare all'interno di un metodo di una classe, il ***this*** continuerà sempre a puntare alla classe stessa

Esempio

Si consideri una classe che espone un metodo ***animate*** il quale esegue una animazione sul DOM al termine della quale si vuole ricolorare lo sfondo di rosso.

- Se la callback di animate viene scritta utilizzando le funzioni tradizionali, all'interno dell'evento il this viene ridefinito e dunque punterà al tag _div

```
class User {
    constructor() {}
    nome="pippo"
    visualizza() {
        let _div =document.querySelector("#myDiv")
        _div.animate(newPos, 1000, function(){
            // this viene ridefinito e rappresenta _div
            this.style.backgroundColor = "red"   //  OK perché this === _div
            this.nome="pluto"                    // NOK perché this === _div
        })
    }
}

let user = new User()
user.visualizza()        // funziona correttamente
console.log(user.nome)   // pippo  nok
```

- Se invece la callback viene scritta utilizzando le arrow function, all'interno dell'evento il this **NON** viene ridefinito e dunque continuerà a puntare alla classe:

```
class User {
    constructor() {}
    nome="pippo";
    visualizza() {
        let _div =document.querySelector("#myDiv")
        _div.animate(newPos, 1000, () => {
            // this NON viene ridefinito e punta alla classe
            this.style.backgroundColor = "red"    // NOK  this === class
            this.nome="pluto"                     //  OK  this === class
        })
    }
}

let user = new User()
user.visualizza()        // la callback finale non ricolora di rosso
console.log(user.nome)   // pluto  ok
```

Per impostare lo sfondo rosso si può comunque utilizzare il solito ***event*** automaticamente iniettato a tutti gli eventi javascript:

```
        _div.animate(newPos, 1000, (event) => {
            event.target.style.backgroundColor = "red"
```

Nota: All'interno di una generica funzione `this` rappresenta un generico "Spazio delle funzioni" del '*global object*' window, per cui quando si accede ad un campo (`this.name`) il risultato sarà sempre `undefined`

I Metodi Funzionali

In ES6 sono state introdotte molti *'metodi funzionali'* per svolgere operazioni su un gruppo di elementi. Queste funzioni sostituiscono i cicli, le ricerche, etc e forniscono prestazioni ritenute migliori.

Metodi di programmazione funzionale applicabili a vettori enumerativi e stringhe

Tutti i metodi seguenti applicati ad un vettore enumerativo richiamano **n** volte la funzione anonima passata come parametro iniettando di volta in volta uno dei record (o valori scalari) costituenti la collezione.
Si tratta di un approccio funzionale alternativo più veloce rispetto alla tipica scansione di un vettore enumerativo con un ciclo FOR OF.

Il metodo `forEach( )`

Applicato a un vettore esegue automaticamente la scansione dell'intero vettore. **A differenza di FOR OF che è applicabile a qualsiasi elemento iterabile, FOR EACH è applicabile SOLO agli elementi di tipo Array.** Riceve come parametro una funzione di callback richiamata in corrispondenza della scansione di ogni singolo elemento. A questa funzione vengono iniettati sia il valore dell'elemento sia il suo indice (facoltativo). **I parametri sono passati per valore, dunque se modifico item, i nuovo valori NON saranno visibili al di fuori del metodo medesimo.** Il metodo forEach è asincrono e non ritorna nulla.

```
vect.forEach(function(item, i) {
      console.log(item)
})
```

I metodi `find()` e `findIndex( )`

Sono molto simili. Entrambi eseguono una scansione del vettore individuando il primo record che soddisfa alla condizione impostata. Però:

- **findIndex** restituisce la **posizione** della prima occorrenza che soddisfa la condizione (-1 in caso di record non trovato)
- **find** restituisce **l'intero record** della prima occorrenza che soddisfa la condizione (**undefined** in caso di record non trovato)

Esempio: Sia **students** un vettore enumerativo di studenti aventi ciascuno una **matricola**

```
let pos = students.findIndex(function(student) {
      return student.matricola == 'AL636GZ'; });
students.splice(pos, 1)
```

findIndex è più completo rispetto al tradizionale **indexOf**.
indexOf consente di ricercare un intero record all'interno di un vettore di record (ovviamente se i puntatori coincidono), mentre **findIndex** consente di eseguire la ricerca sulla base di uno o più campi

Il metodo `filter( )`

Restituisce un vettore enumerativo contenente **tutti** i record che soddisfano al criterio di filtro:

```
let vet = students.filter(function (student) {
      return student.residenza == "Fossano"
})
```

Notare che **`filter`** NON restituisce una copia, ma contiene gli STESSI oggetti del vettore originale solo filtrati. **Entrambi gli array puntano agli stessi riferimenti in memoria** !

`filter` rappresenta una interessante alternativa allo splice per eliminare un record:

```
students = students.filter(function(x) {
      return student.matricola != 'AL636GZ'; })
```

Il metodo `map ( )`

Il metodo **map** applica la funzione anonima ricevuta come parametro ad ogni elemento dell'array restituendo un **nuovo array** avente in linea di massima **la stessa lunghezza** dell'array iniziale, in cui ogni elemento è il risultato dell'elaborazione sull'elemento corrispondente nell'array iniziale. Note:

- L'array iniziale non viene modificato
- La funzione non viene eseguita su eventuali celle prive di valore

```
let materials = ["Hydrogen", "Helium", "Lithium", "Beryllium"];

let lengths = materials.map(function(item) {        // [8, 6, 7, 9]
    return item.length;
});
```

// Invece di una funzione anonima si può passare direttamente una funzione esistente che verrà richiamata per ogni item e a cui verrà iniettato l'item stesso :

```
let numbers = [4, 9, 16, 25];
let ris = numbers.map(Math.sqrt);     // [2, 3, 4, 5];
```

Il seguente esempio restituisce un vettore enumerativo contenente gli ID di tutti i checkbox della pagina.
Il metodo querySelectorAll restituisce una NodeList a cui non è applicabile il metodo .map()

```
let ids = Array.from(document.querySelectorAll("input[type=checkbox]"))
        .map(function(item) {
              return item.id;
         })
```

Per fare in modo che, ad ogni iterazione, map restituisca un JSON si può avvolgere il corpo della funzione con delle parentesi tonde:

```
const names = vet.map((item, i) => ({nome : item, index : i}) )
```

Spread Operator …

La sintassi **Spread** (indicata tramite tre puntini consecutivi) permette di suddividere un vettore enumerativo o un vettore associativo nei suoi elementi costitutivi.

Array enumerativi

```
let parts = ['mele', 'pere']
let fruits = ['arance', 'ciliegie' ...parts, , 'uva']
// risult  = ['arance', 'ciliegie' 'mele', 'pere', 'uva']
```

Attenzione però che se uno degli elementi è a sua volta un vettore annidato, questo viene restituito così com'è.

```
let parts  =  ['a', 'b', ['c', 'd']]
  ...parts  ->  'a', 'b', ['c', 'd']
```

Altro esempio: calcolo del massimo all'interno di un vettore di numeri:

```
const numbers = [1, 2, 3, 4, ];    let max = Math.max(...numbers)
```

Array associativi (json)

Applicato ad un json, restituisce singolarmente tutte le coppie chiave/valore:

```
let obj1 = { foo:'bar', x:42 };
let obj2 = { foo:'baz', y:13 };
let  mergedObj = { ...obj1,  ...obj2 };
// result =     { foo: "baz", x: 42, y: 13 }
```

Il secondo valore di `foo` sovrascrive il primo (non possono infatti esistere due chiavi uguali)

Lo spread operator consente anche di clonare un json esistente

```
let clonedObj = { ...obj1 };
```

Questa istruzione è simile a clonedObj = obj1 però, mentre clonedObj = obj1; copia semplicemente il puntatore, l'utilizzo dello spread operator **crea un nuovo oggetto**.
Attenzione però che nel caso di oggetti annidati viene copiato il puntatore, per cui se modifico una property annidata in un oggetto, la modifica si ripercuote anche sull'altro oggetto.

Lo Spread Operator è molto comodo anche per convertire una NodeList javascript (o una collezione jQuery) in un vettore enumerativo puro, come ottima alternativa al metodo Array.from() :

```
let vet = [...NodeList]
```

Il metodo `Array.from ( )`

Il metodo Array.from() converte in Array l'oggetto passato come parametro (ad esempio una NodeList).
Notare che **restituisce un nuovo oggetto**, quindi re inizializza tutti i puntatori.

Esiste però anche una seconda firma costituita da due parametri dove:

- Il primo parametro è un vettore enumerativo iterabile
- Il secondo parametro è una funzione di map che agisce sul primo vettore e restituisce un nuovo vettore

```
console.log(Array.from( [1, 2, 3],  function(x) { return x*2 } )) //[2, 4, 6]
```

La funzione di callback in realtà può ricevere due parametri : il valore e l'indice (facoltativo)

```
Array.from( [1, 2, 3],  function(x, i) { return x*2 } )
```

Il primo parametro è obbligatorio. Se non si intende farne uso, si assegna normalmente il nome underscore

```
Array.from( [1, 2, 3],  function (_, i) { console.log(i) } )
```

Creazione di un vettore ex-novo

Come primo parametro si può anche passare un json indicante la lunghezza del vettore.
Il seguente esempio crea una stringa casuale di 20 caratteri minuscoli:

```
let str = Array.from({length:20},
                     function(){return String.fromCharCode(random(26,98))})
         .join('')
```

// oppure in modo 'tradizionale' :

```
let str [...(new Array(20))].map( function(){
  String.fromCharCode(random(26,98)) } ).join('')
```

`new Array(20)` crea un array di lunghezza 20, ma con tutti elementi "vuoti" (<empty>) **non iterabile.**
Le posizioni esistono ma non sono definite
Lo **`spread operator ...`** crea 30 elementi lineari tutti undefined e Le **`quadre`** `finali` ricostruiscono il vettore che sarà ora un vettore reale (quindi iterabile) con 20 celle reali tutte contenenti undefined.

Notare che in javascript richiamare Array() con o senza **`new`** NON cambia assolutamente nulla.
Il risultato è sempre lo stesso: un array di lunghezza 20 con slot vuoti (<empty>) non iterabile.

La seguente riga estrae 10 celle diverse da un vettore iniziale questions :
Array.from({length:10}, () => questions.splice(random(0, questions.length), 1)[0])

L'oggetto Set

Gli oggetti SET sono delle '**collezioni' iterabili**, sostanzialmente vettori enumerativi, **in cui ogni valore può ripetersi una ed una sola volta.** Possono contenere qualsiasi item (scalari, stringhe, json, vettori).

Se si trasforma un vettore in oggetto Set, **tutte le voci replicate vengono rimosse**

```
let vet = [5, 6, 7, 6, 8, 6, 9, 7]
let set = new Set(vet)

set.add(9)            // non fa nulla perché il 9 è già presente
console.log(set)      // Set(5) { 5, 6, 7, 8, 9 }
```

A questo punto si può ritrasformare l'oggetto SET in normale vettore : `let vet2 = [... set]`

.***size*** restituisce la dimensione del SET
.***clear()*** svuota il SET
E' possibile scorrere normalmente l'oggetto SET con un ciclo ***FOR-EACH***.
Non sono invece supportati gli altri metodi funzionali, nel qual caso occorre convertire il SET in Array

Il metodo `sort( )`

Dato un vettore di JSON, per ordinarlo si può utilizzare il metodo **`.sort()`** specificando come parametro il nome del campo su cui eseguire l'ordinamento, ed anche il verso crescente/decrescente.
La seguente procedura esegue un ordinamento del vettore sulla base del nome dei singoli studenti:

```
students.sort(function(item1, item2) {
        let str1 = item1.nome.toUpperCase();
        let str2 = item2.nome.toUpperCase();
        if (str1 < str2)
            return -1;
        else if (str1 > str2)
            return 1;
        else return 0;
    });
```

Per fare l'ordinamento inverso è sufficiente invertire il return all'interno delle due IF

Questa tecnica può essere usata per 'mescolare' casualmente il contenuto di un vettore (anche se in modo non troppo efficiente) :

```
vet.sort(function (a, b) {return Math.random() - 0.5);
```

`Math.random()` restituisce un numero decimale tra 0 e 1. Se è > 0.5 inverte, altrimenti NON inverte.

Il metodo `.replace( )`

Applicato ad una stringa sostituisce la sottostringa contenuta nel primo parametro con quella contenuta nel secondo parametro. E' anche possibile:

- Sostituire un **intero JSON** con un altro JSON (solo se il puntatore del primo JSON coincide con il puntatore all'interno del vettore)
- passare come primo parametro una espressione regolare che indica tutte le occorrenze da sostituire con il secondo parametro

Inoltre in **ES6** è possibile passare come 2° parametro una funzione di callback alla quale, ad ogni scansione, viene iniettato come parametro una della occorrenze individuate dall'espressione regolare impostata come primo parametro. Questa funzione può elaborare / modificare ciascuna occorrenza ritornando ad ogni scansione il nuovo valore.

```
str.replace(/\b\w/gi,    function (m) { return m.toUpperCase(); });
str.replace(/\b[a-z]/gi, function (m) { return m.toUpperCase(); });
```

equivalenti. Convertono in maiuscolo la lettera iniziale di ciascuna parola contenuta all'interno della stringa

La seguente riga invece evidenzia tutte le occorrenze della parola indicata.

```
let parolaDaCercare = "……"
let regEx = new RegExp(parolaDaCercare, "gi");
textBox.value = textBox.value.replace(regEx, (match) => {
  return `<mark>${match}</mark>`})
```

Per eliminare invece l'evidenziatore:

```
let regEx = new RegExp("<mark>|</mark>", "gi");
textBox.value = textBox.value.replace(regEx, (match) => {return ""});
```

Punto interrogativo ? e ??

```
div?.style.color="red"  // se div è undefined l'istruzione NON viene eseguita

let altitude = data.altitude ?? "Non disponibile";
```

Se `data.altitude` è NULL or UNDEFINED, allora altitude = "Non disponibile".
?? è più selettivo rispetto a ||. Fa scattare la condizione SOLO su NULL e UNDEFINED

```
// L'istanza viene creata SOLO se chart non esiste
chart ??= new Chart()
```

Il metodo `.flat()`

Trasforma un vettore di vettori in un vettore con tutti gli elementi al primo livello. Può agire anche su più livelli di annidamento.

```
const vet = [0, 1, 2, [3, 4]];
console.log(vet.flat());      // [0, 1, 2, 3, 4]
```

Comodo ad esempio quando vari oggetti presentano un campo vettoriale comune (es hobbies) e si vuole ottenere un vettore con tutti i valori di quel campo (vettore che potrà poi essere reso univoco tramite Set)

```
let hobbies = []
for(let student of students)
  hobbies.push(student.hobbies)   // push di un intero vettore
hobbies = hobbies.flat()
let set = new Set(hobbies)
```

Il metodo `.flatMap()`

Identico a .map() ma in più esegue un flat() finale (nel caso in cui la map() sostituisca una cella scalare con un piccolo vettore enumerativo).

Il metodo `some( )`

Restituisce **`true`** se **almeno un elemento dell'array** soddisfa ai criteri della condizione impostata.
Evita un ciclo for sull'array. Non modifica l'array.

```
const array = [1, 2, 3, 4, 5];
console.log(array.some(function (item) {
     return item%2 == 0
})
```

Object.freeze(par)

Object.freeze(par) rende immutabile il parametro ricevuto. Accetta stringhe, vettori, oggetti

typeof e istanceof

La funzione **typeof**(myVar) restituisce come stringa il tipo di una **variabile scalare**. Esempio:

```
if (typeof(myVar)== "number"  ||  typeof(myVar)== "string"  )
```

Applicata **però** ad una variabile di tipo Oggetto, la funzione **typeof(myVar)** restituisce **sempre** "**object**".
(questo anche nel caso di un vettore).

Per verificare di quale object si tratta, si può utilizzare l'operatore **instanceof** che, a differenza di typeof(), non è una funzione ma un semplice operatore come ad esempio ==

```
if (myVar instanceof Array) ...      // oppure if(Array.isArray(myVar))
if (myVar instanceof Object) ...     // json
if (myVar instanceof FormData) ...
```

Oppure si può utilizzare la seguente istruzione che restituisce l'object type come semplice stringa:

```
if (myVar.constructor.name == "FormData")
```

Il metodo animate()

```
const animation = element.animate(
  { transform: "translateX(600px)"},
  { duration:3000, fill:"forwards"})
await animation.finished;     // restituisce una promise risolta al termine dell'animazione
animation.commitStyles();
//animation.cancel();
```

- Il **primo parametro** indica la proprietà CSS da animare.
  E' possibile animare qualsiasi proprietà css numerica. Anche più di una contemporaneamente :

```
id1.style.position = "relative"; // o "absolute"
id1.style.left = "0px";  // valore iniziale obbligatorio
const animation = id1.animate(
  { left:"600px", "marginTop":"80px"},
  { duration:3000, fill:"forwards"})
await animation.finished;
animation.commitStyles();
//animation.cancel();
```

  E' anche possibile animare più property in sequenza, passando come prima parametro un vettore enumerativo di json, ciascuno con un proprio offset temporale. Se l'offset non viene specificato, viene equamente suddiviso fra i vari valori.

```
element.animate( [{opacity:1}, {opacity:0.1, offset: 0.7}, {opacity:0}],2000)
```

- Il **secondo parametro** può essere semplicemente un tempo in sec oppure si può specificare un secondo json come indicato sopra. L'opzione **fill:"forwards"** serve a rendere persistente l'animazione, nel senso che l'oggetto rimane nella sua posizione finale, altrimenti ritornerebbe alla posizione iniziale. Insieme a fill:"forwards" è consigliato l'utilizzo di **animation.commitStyles()** che va a settare le nuove property CSS.

animation.cancel() consente di riportare l'oggetto nella sua posizione iniziale antecedente l'animazion

fadeIn

```
div.style.transition='opacity 1s';
function esegui() { div.style.opacity=0;
                    setTimeout(() => div.style.opacity=1, 1000)   }
```

Delegated Events

Considerazioni sulla associazione degli eventi

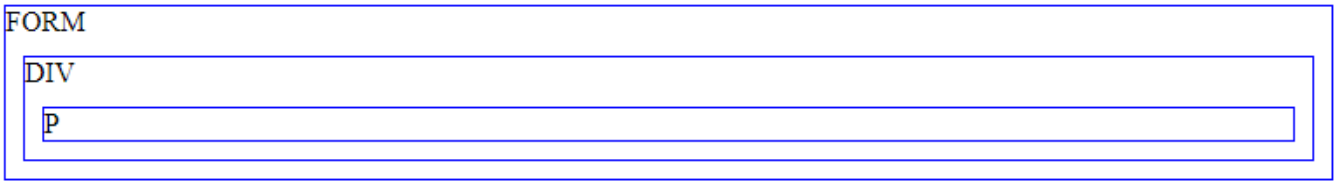
A differenza delle proprietà CSS definite all'interno di un foglio di stile che vengono applicate anche a posteriori sui nuovi elementi creati dinamicamente, **l'associazione degli eventi ad un elemento del DOM deve essere fatta soltanto DOPO che l'oggetto è stato creato**.

Per gli elementi creati dinamicamente di solito si associa l'evento durante la creazione stessa. Se però un medesimo addEventListener viene eseguito più volte su uno stesso elemento, nel caso delle procedure anonime l'associazione di evento viene eseguita più volte, per cui poi l'evento si verificherà più volte.

I **delegated Events** rappresentano una interessante possibilità per associare gli eventi in fase di avvio anche per elementi non ancora esistenti. La tecnica consiste nell'associare gli eventi ad un elemento esistente, per poi **delegarli agli elementi interni** (esistenti o che verranno creati dinamicamente in un secondo tempo)

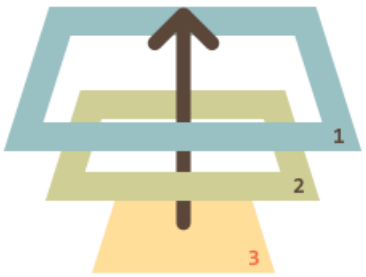
Funzionamento degli eventi in javascript : Directly bound events

Si consideri la seguente struttura html




```
<form onclick="alert(this.name)">
   <div>
      <p> testo </p>
   </div>
</form>
```

Se si clicca **su un qualsiasi elemento interno** alla form, la alert() viene sempre eseguita. Quando infatti si verifica un evento su un elemento interno, **questo evento viene sempre propagato verso l'alto** (**Event Bubbling**) a tutti i suoi genitori, fino ad arrivare al tag ***form*** e poi anche al tag ***body***. Ciascun elemento della catena può gestire o non gestire l'evento, però l'evento viene comunque sempre propagato.



L'aspetto interessante è che, **associando l'evento al tag form che esiste fin dall'avvio**, l'evento si verificherà anche nel caso di **elementi interni aggiunti dinamicamente in un secondo tempo.**

Però attenzione che all'interno dell'evento, *this* rappresenta SEMPRE la form, indipendentemente da dove si è cliccato !

Se aggiungiamo un gestore di evento anche sugli elementi interni

```
<form onclick="alert('form')">                  // (this = form)
   <div onclick="alert('div')">                 // (this = div)
       <p onclick="alert('p')"> testo </p>      // (this = p)
   </div>
</form>
```

Se si clicca su tag P compaiono i messaggi **p div form** *(bubbling phase)*
Se si clicca su tag DIV (fuori da P) compaiono i messaggi **div form**
Se si clicca sulla form (fuori da DIV) compare il messaggio **form**

Delgated Events

Si supponga di gestire l'evento soltanto sulla form esterna

Quando si clicca su un elemento interno, insieme all'evento, **viene SEMPRE propagato anche il puntatore all'elemento che ha scatenato l'evento**, puntatore che sarà **disponibile all'interno di *event.target*** dove ***event*** viene automaticamente iniettato a tutti gli eventi javascript.

```
form1.addEventListener("click", function(event){
     if(event.target.matches("p")   // .matches è un metodo disponibile sugli element del DOM
         alert(event.target.textContent)
})
```

event.target puntatore all'elemento che ha scatenato l'evento.
event.currentTarget puntatore all'elemento primario (esterno) a cui è collegato l'evento (cioè ***form***)

Si dice che la form **delega** la gestione dell'evento all'elemento interno su cui l'evento si è verificato *(capturing phase).*

Vantaggi:

- **Al momento della definizione dell'evento è sufficiente che esista il contenitore esterno**. **Gli elementi interni** (che possono essere aggiunti dinamicamente anche in un secondo tempo) **delegano la gestione dell'evento all'elemento esterno pre-esistente.**

- Un altro notevole vantaggio è legato alle prestazioni. Supponendo di avere una tabella con 1000 righe, il seguente codice definisce 1000 associazioni di evento, una per ogni **tr**

```
let rows = document.querySelectorAll("tr")
for (let tr of rows)
    tr.addEventListener( "click", function() {
        console.log(this.textContent)  })
```

Il seguente codice definisce invece **una sola associazione di evento** su **table** demandando poi la gestione dell'evento agli elementi interni:

```
table.addEventListener( "click", function(e) {
     if (e.target.matches("tr"))       // if(e.target.tagName == "tr")
         console.log(e.target.textContent)
});
```

Svantaggi:

- Se siamo dentro un ciclo che scorre i record di un vettore di json e li visualizza uno per riga, se l'evento è definito al form_load non posso passare il record corrente all'evento, record che dovrà quindi essere memorizzato in forma serializzata all'interno di un data-attribute del TR corrente.

Le classi in java script

Concetto di Classe

- JS per sua natura non è un vero e proprio linguaggio Object Oriented, il che lo rende molto più leggero.
- In java script **le classi sono semplicemente "funzioni speciali" che consentono di "creare" un JSON** mediante una sintassi alternativa rispetto alla scrittura diretta del json stesso.

  I **vantaggi** relativi all'utilizzo delle classi rispetto alla scrittura diretta di un JSON sono:
  - La possibilità di gestire un controllo più fine sull'assegnazione dei valori
  - La possibilità di creare **più** JSON diversi a partire da una stessa funzione "costruttrice"

Dichiarazione delle classi in ES5

Le classi in javascript vengono definite tramite una normale funzione che però deve essere invocata NON direttamente ma tramite l'operatore **new** che, letteralmente, è quello che crea il json.

Quando una normale funzione sarà utilizzata con il **new** come generatrice di json, al suo interno si può utilizzare il this per creare le chiavi del json:

```
let User = function(name, surname){
   this.name = name;                        // creo una Property name
   this.surname = surname;                  // creo una Property surname
   this.setName=function(s){this.name=s};   // creo un method
}
let user = new User("Henry", "Ford");
```

Il new, quando invocato**, crea un nuovo JSON vuoto e fa sì che, all'interno della funzione User, il `this` punti a questo json. Tutti i campi creati con l'operatore `this` definiscono una chiave all'interno del json, i cui campi assumono il significato di Proprietà e Metodi della classe.**

In pratica la funzione User() è come se fosse il **costruttore** dell'Oggetto User

Note:

- Se si omette il **`this`** davanti al nome del campo, il campo diventa sostanzialmente una variabile PRIVATA all'interno della funzione e non viene creata nessuna property
- **`User,`** essendo una normale **Function,** di per se può anche essere richiamata normalmente **senza il new**. Però, nel momento in cui viene richiamata senza il new, **`this`** risulterà undefined, per cui tutte le istruzioni che iniziano con il this andranno in syntax error

L'istruzione Prototype

Gli Oggetti istanziati tramite classe sono **estensibili, cioè diventa possibile in qualunque momento estendere la classe (mediante la parola chiave `prototype`) con l'aggiunta di nuovi metodi che saranno automaticamente accessibili da tutte le istanze**

```
User.prototype.sayHi = function() {
    alert(this.name);
}
user.sayHi();
```

Il prototype deve essere applicato al **nome della classe** ed è poi utilizzabile in tutte le istanze.

Note

1. I metodi per accedere ai membri della classe (proprietà e metodi) devono sempre utilizzare il **`this`**
2. I metodi, oltre che accedere alle property e agli altri metodi della classe, possono anche accedere a semplici funzioni definite nello stesso file al di fuori della classe. In questo caso occorre omettere il this. Conviene inserire all'interno della classe tutti i metodi che devono essere visibili all'esterno (**metodi pubblici**) e definire come semplici funzioni tutti quei metodi che devono essere utilizzati solo all'interno della classe (**metodi privati**).
3. A livello operativo definire un metodo "all'interno della classe" oppure "all'esterno tramite prototype", NON cambia assolutamente nulla. Però:
   - I metodi definiti tramite il **this** all'interno della classe (es. `setName()`) vengono replicati all'interno di ogni singola istanza.
   - I metodi definiti tramite **prototype** **non** vengono replicati all'interno di ogni singola istanza**.** Vengono invece memorizzati all'interno di un'area comune; a tutti gli effetti faranno parte delle istanze, (e dunque potranno accedere ai membri di istanza tramite il this), ma il loro codice non viene replicato in tutte le istanze. Un po' come avviene normalmente per i metodi nella programmazione Obiect Oriented

   Un altro aspetto importante nell'utilizzo del prototype è che tutte le modifiche apportate al prototype vengono propagate anche agli oggetti già allocati.

Dichiarazione delle classi in ES6

ES6 introduce una nuova sintassi per la gestione delle classi tramite la parola chiave **`class`** che rappresenta una semplice **sintassi alternativa** a quella già esistente in ES5 e NON un nuovo modello di ereditarietà orientata agli oggetti, come nei linguaggi Object Oriented.
La parola chiave **`class`** è semplicemente un 'abbellimento estetico' della sintassi precedente.

Come in ES5 **le classi sono semplicemente "funzioni speciali" che, al momento del new, consentono di "creare" un JSON** mediante una sintassi alternativa rispetto alla scrittura diretta, consentendo di fatto un maggior controllo sui tipi e sugli accessi R / W

In ES6 la classe precedente può essere riscritta nel modo seguente:

```
class User {
  name=""           // property
  #password=""      // property privata
  constructor(name) {
     this.name = name;
  }
  sayHi() {
     alert(this.name);
  }
}

let user = new User("John");
console.log(user.name)
user.sayHi()
```

- All'interno della classi si possono definire sia **proprietà** sia **metodi**.
  Entrambi possono facoltativamente essere terminati da punto e virgola
- **Davanti alle Property occorre omettere sia `var` sia `let`.**
- **Davanti ai Method occorre omettere `function`**
- Non sono ammessi separatori fra i campi di una classe, nemmeno la virgola.

- Il codice scritto all'interno di una classe viene automaticamente gestito in **`strict mode`**, cioè è obbligatoria la dichiarazione delle variabili.
- In javascript davanti a proprietà e metodi non sono ammessi i **qualificatori di visibilità** public e private che per default sono tutti **public**. In ES2022 è stato introdotto un nuovo simbolo il # che, anteposto al nome d una Property di un Method; consente di renderlo private : **`#password`.** Attenzione che il # entra a far parte del nome della property, che dunque, all'interno della classe, dovrà essere acceduta come #password.
- La dichiarazione esplicita delle **Property** non è obbligatoria. Se nell'esempio precedente si omettesse la Property **`name`**, essa verrebbe automaticamente creata dal costruttore in corrispondenza dell'istruzione `this.name = name`
- **I metodi** (essendo dichiarati internamente mediante prototype) **non** vengono replicati all'interno di ogni singolo JSON ma vengono invece memorizzati all'interno di un'area comune della funzione in modo che tutte le istanze possano accedervi.

Traduzione della classe precedente in codice ES5

In pratica la parola chiave **class** fa due cose:

- **Definisce una funzione avente il nome della classe e che rappresenta un riferimento alla funzione definita tramite la parola chiave constructor**
- Crea una funzione .prototype per ogni metodo elencato all'interno della classe.

```
let User = function (name){
   this.name = name;
}

User.prototype.sayHi = function() {
   alert(this.name);
}
```

```
// main
let user = new User("John");
console.log(user.name)
user.sayHi();
```

Note:

- A differenza di ES5, una funzione definita tramite **class** non può essere richiamata senza il new. Il **new**, come detto, è quello che fisicamente crea e restituisce il JSON con all'interno tutte le property definite all'interno della classe.
- Se all'interno della classe non si specifica il metodo costruttore, viene automaticamente inserito un costruttore vuoto, come se avessimo scritto `constructor() {}`
- Anche nella sintassi ES6 continua ad essere consentito l'utilizzo del **prototype**

```
class User { }
let user = new User()
User.prototype.test = 5;
alert(user.test);    // 5
```

Metodi Statici

I metodi statici possono essere definiti direttamente all'interno della classe tramite la parola chiave **static**

```
class User {
  static staticMethod() { alert("Static Method");  }
}
```

Oppure anche "fuori" dalla classe nel modo seguente (senza il prototype):

```
User.staticMethod = function () { alert("Static Method") };
```

Il richiamo deve avvenire mediante il nome della classe:

```
User.staticMethod();
```

Definizione delle Property di una classe tramite Getters / Setters

Per definire le property della classe è anche possibile utilizzare i metodi **getter** e **setter** che servono per controllare gli accessi alle property. Notare come sia **get** che **set** trasformano in Property quello che in apparenza è un metodo (**name**)

```
class User {
   constructor(name) {
      this._name = name;    // se esiste, invoca il setter
   }
   get name() {
      return this._name;
   }
}

   set name(value) {
     if (value.length < 4)
        alert("Name is too short.");
     else
       this._name = value;
   }
```

```
let user = new User("John");
alert(user.name); // John
user.name = 'Jack'
user = new User(""); // Name is too short.
```

Struttura delle librerie javascript

Storicamente esistevano in javascript due diversi standard per l'import di una libreria:

- Lo standard **UMD** (Universal Module Definition) usato lato **client** con link del modulo direttamente dal **file.html** tramite utilizzo del tag **`<script src= "……"> </script>`**
- Lo standard **CommonJS** usato lato **server** e caratterizzato dall'utilizzo dei metodo **`require()`**

Con ES6 (2015, ma supportato dai browser solo dal 2018) è stato introdotto un nuovo standard detto **ES-Modules** utilizzabile sia lato client che lato server, che ha poco alla volta rimpiazzato i due standard precedenti, soprattutto lato server. Tre le differenze principali :

1. In UMD tutti i files linkati nell'html si vedono fra loro come se ci fosse un unico grande file, ESM è invece di tipo modulare e gestisce i moduli come veri e propri namescape con un loro scope. Cioè **i vari moduli non si vedono fra loro ed ogni modulo vede SOLO ciò che importa**.
2. Mentre i due predecessori erano entrambi strutturati in modo **sincrono** (blocco dell'applicazione fino al termine del caricamento del modulo), ES-Modules carica i moduli dinamicamente in modo **asincrono** con una callback che rende disponibile il modulo all'applicazione nel momento in cui serve. Molto più prestante.
3. Con i due predecessori occorreva importare sempre l'intera libreria, mentre ES-Modules consente di importare la libreria intera oppure soltanto **i singoli metodi** che servono in modo da non appesantire troppo l'applicazione.

Import di un modulo ESM in vanilla javascript

- Una libreria **ESM** NON può essere importata direttamente in file html tramite utilizzo del solito tag **`<script src= "……"> </script>`**
  ma DEVE essere importata da un altro file.js (ad esempio index.js) mediante l'istruzione **import**
- L'istruzione **`import`** può essere utilizzata SOLTANTO **all'inizio del file.js** prima di qualsiasi altra istruzione
- Un file index.js che richiama un modulo ESM tramite un'istruzione **import,** DEVE essere dichiarato nell'html con l'opzione **`type='module'.`**
  `<script type="module" src="./index.js"></script>`
- Se più file.js dichiarati con **`type='module'`** necessitano di dover utilizzare una stessa libreria, ogni file dovrà importarla per conto proprio.

- i moduli ESM, per motivi di sicurezza, funzionano **solo** se la pagina viene servita da un **server HTTP** (non tramite file://). Quindi, se si apre il file HTML direttamente con doppio click, si ottiene un errore del tipo: "*Access to script at ... from origin 'null' has been blocked by CORS policy*" .

  Due semplicissimi modi per avviare un http Server sono :
  - usare **npx** che è una utility di nodejs (installata a livello globale insieme a node) da eseguire a linea di comando per lanciare il comando ***serve*** il quale avvia un http server : ***`npx serve .`***
    (porta 3000) http://localhost:3000

- Installare in Visual Studio Code l'utility **Live Server** di Ritwick Dey che può essere eseguito in modo contestuale (con tasto destro in qualsiasi punto del file html) oppure tramite apposto comando in basso a destra sulla barra di stato (**Go Live**, *porta 5500 http://localhost:5500*)

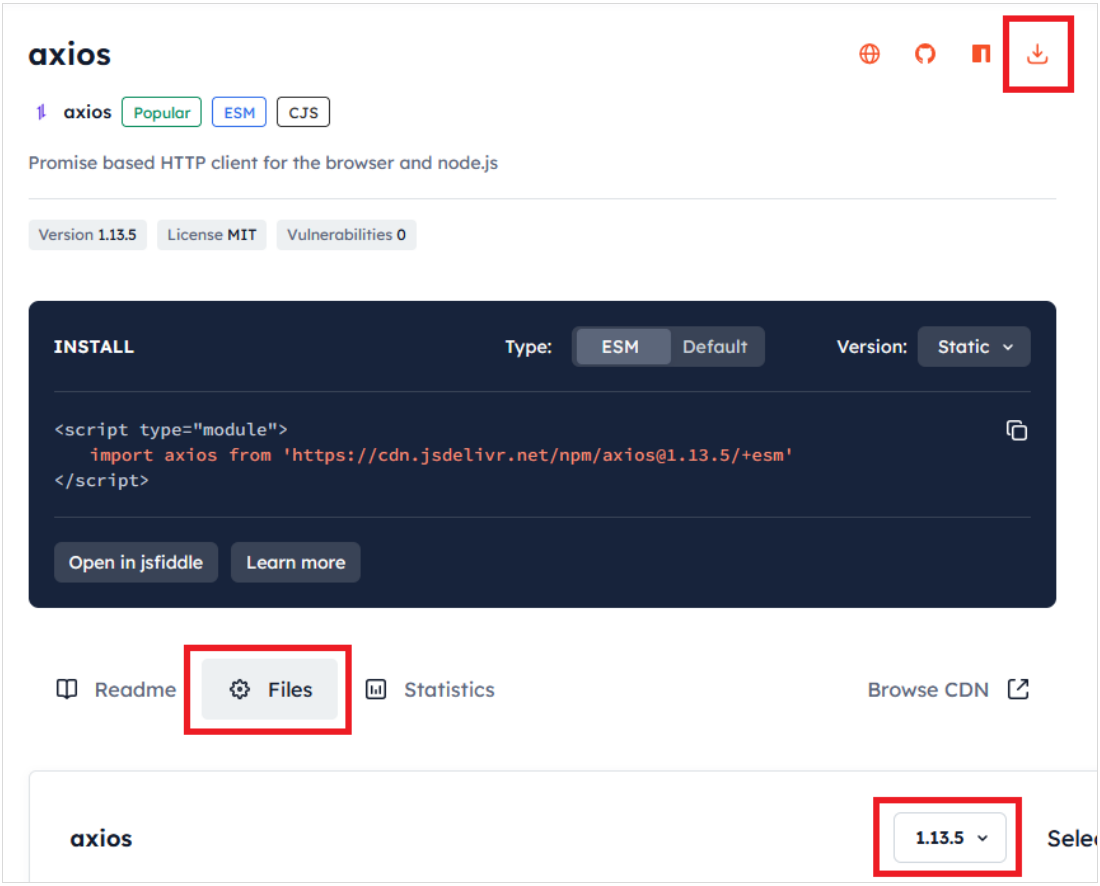
Distribuzione delle librerie

- Se si intende utilizzare una libreria in modo 'tradizionale' con link dall'html occorre scaricare la versione **UMD** (o utilizzare un CDN Content Delivery Network, Rete di Distribuzione dei Contenuti)
- Se si intende utilizzare la libreria come 'modulo' con link dal file javascript occorre scaricare la versione **ESM** (o utilizzare un relativo CDN). Per in questa modalità occorre un server http.

I due principali CDN di distribuzione delle librerie javascript sono **cdnjs/cloudflare** e **jsDelivr** che rendono disponibili sia il download sia il CDN.

jsDelivr

Scrivendo ad esempio **`jsdelivr axios`** si apre la seguente finestra :



In cui l'utente può:

- Scegliere la versione (in basso a destra dopo aver selezionato **Files**)
- Scaricare l'intero **npm package** (in alto a destra)
- Copiare il link **UMD** (indicato dalla voce ***Default***)

```
<script src="https://cdn.jsdelivr.net/npm/axios@1.13.2/dist/axios.min.js">
</script>
```

- Copiare il link **ESM** che però per come è scritto trae un po' in inganno

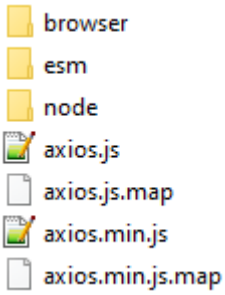
```
<script type="module">
   import axios from 'https://cdn.jsdelivr.net/npm/axios@1.13.5/+esm'
</script>
```

La sintassi è corretta, però importando axios in questo modo, poi non potrà essere utilizzato dentro index.js. L'intero codice di utilizzo di axios dovrà essere scritto all'interno dello script.
L'approccio corretto è invece quello di richiamare dall'html **`index.js con type="module"`** e poi eseguire l'import direttamente all'inizio di index.js
Il **`+esm`** finale è corretto e viene interpretato direttamente da jsdelivr

npm package

Il download viene di solito distribuito come **package npm**, che è il formato utilizzato da nodejs.
In figura è riportata l'alberatura della libreria ***`axios@1.13.2`***.
Nella sottocartella **`/dist`** si trova la libreria **UMD** disponibile nelle due versioni completa e minimizzata

Alberatura della cartella `/dist` :



Una volta scaricato il package npm sono possibili due strade:
- Copiare nel progetto il solo file UMD axios.js
- Allegare al progetto l'intero package (più pesante) e poi aggiungere il link **`axios/dist/axios.js`**

Nell'ulteriore sottocartella **`/esm`** di trova la versione ESM sempre in versione completa e minimizzata

Il comando npm install

Se si conosce il nome della libreria da installare (ad esempio axios), è sufficiente lanciare dalla cartella di lavoro il comando

```
npm install axios
```

il quale scarica dal repository npm l'ultima versione del **package** indicato, salvandola all'interno di una speciale sottocartella denominata node_modules. Nel file occorrerà indicare il path completo:

```
<script src=node_modules/axios/dist/axios.js>
```

Il nome della libreria viene anche aggiunto all'interno di un file denominato package.json contente l'elenco di tutte le librerie utilizzate nel progetto.

Il file .map

Al di là delle minimizzazioni (a capo, indentazioni e commenti) le librerie sono spesso scritte usando un codice ottimizzato ma poco leggibile. Il file **.map** abbinato alla libreria è una **source map**, cioè una "mappa" che collega il codice trasformato al codice sorgente originale, in modo da consentire un più agevole **debugging** del codice stesso. Ad esempio

```
function(a,b,c){return a.b(c)?...}
```
diventa

```
function getData(id) {
    return fetch(`/api/${id}`);
}
```

Il file ***.map*** viene collegato alla libreria in modo automatico senza necessità di ulteriori link.
In coda alla libreria c'è una riga del tipo `//# sourceMappingURL=axios.min.js.map` che dice al browser di scaricare anche il file .map

Salvataggio di un testo nella clipboard

```
await navigator.clipboard.writeText(text);
const text = await navigator.clipboard.readText();
```

La lettura dalla clipboard ha però alcune limitazioni di sicurezza:

- Funziona solo in contesti sicuri (HTTPS)
- Deve essere eseguita in risposta a un'azione dell'utente (es. click)
- Il browser potrebbe chiedere il permesso esplicito

La prassi è quella di gestire la lettura con un TRY - CATCH

Approfondimenti

window.matchMedia

serve a verificare se una media query CSS è soddisfatta e a reagire ai relativi cambiamenti (es. resize della finestra, cambio orientamento, tema scuro, ecc.).

```
const mediaQuery = window.matchMedia("(max-width: 768px)")
```

Restituisce un oggetto ***mediaQuery*** con i seguenti campi:

.matches → true se la media query è soddisfatta
.media → la stringa della media query // `(max-width: 768px)`
.**addEventListener("change", function(mediaQuery)){}** → per intercettare i cambiamenti

```
const mediaQuery = window.matchMedia("(max-width: 600px)")
mediaQuery.addEventListener("change", function(mq) {
   if (mq.matches) {
     console.log("Visualizzazione mobile")
   }
   else {
     console.log("Visualizzazione PC")
   }
})
```

Applicazione di un tema dark-light con bootstrap

$('html').attr('data-bs-theme', theme);

Applicando all'attributo **data-bs-theme** il valore del tema scelto, bootstrap automaticamente applica il tema al'intera pagina. la variabile **theme** può assumere i seguenti valori:

- light
- dark
- auto // legge il tema di sistema

Il css di bootstrap sarà fatto in modo simile a questo (semplificato al massimo) :

```
[data-bs-theme="light"] {
  --bs-body-bg: #002b36;
  --bs-body-color: #839496;
}
```

Nel caso del valore **auto**, bootstrap va a leggere il tema di sistema con il metodo `window.matchMedia`

```
const darkMode = window.matchMedia("(prefers-color-scheme: dark)");
if (darkMode.matches) {
  document.body.classList.add("dark");
}
```

prefers-color-scheme è una media feature CSS che indica la preferenza dell'utente per il tema chiaro o scuro**,** impostata a livello di sistema operativo o browser. **javascript può leggere questo valore ma non può modificarlo**. Serve per applicare (***manualmente***) al sito il tema impostato sul browser (nel caso dei PC) oppure sul Sistema Operativo (nel caso degli smartphone).

requestAnimationFrame()

requestAnimationFrame() è un'API del browser usata per eseguire animazioni e aggiornamenti grafici in modo efficiente e sincronizzato con il refresh dello schermo.

Il browser esegue un refresh dello schermo ad ogni framerate, cioè in pratica 50 volte al secondo (60 negl USA). requestAnimationFrame() chiede al browser di eseguire la funzione ricevuta come parametro in corrispondenza del prossimo refresh

Rappresenta una recente alternativa a setInterval e setTimeout (che non sono sincronizz con il rendering dello schermo e consumano più risorse) e consente di rendere le animazioni il più fluide possibile.

```
function animate() {
  console.log("frame");
  requestAnimationFrame(animate);
}
requestAnimationFrame(animate);
// stampa una volta per ogni frame (~50 volte al secondo).

let x = 0;
function move() {
  x += 2;
  box.style.transform = `translateX(${x}px)`;
  requestAnimationFrame(move);
}
let timerId = requestAnimationFrame(move);
cancelAnimationFrame(timerId);
```

intersectionObserver()

intersectionObserver() è un'API del browser che serve a osservare quando un elemento entra o esce dall'area visibile (***viewport***) della finestra (o di un contenitore) a seguito di uno scroll.
Rappresenta una alternativa più moderna a scroll event che è molto più pesante (richiamato decine di volte al secondo) e presenta un codice di gestione molto più complesso

```
const observer = new IntersectionObserver(function(entries) {
  entries.forEach(entry => {
    if (entry.isIntersecting) {
      console.log("Elemento visibile");
    }
  });
})
observer.observe(document.querySelector("#box"));
```

`entries` è un array di oggetti *IntersectionObserverEntry* che il browser passa automaticamente alla callback, uno per ogni elemento osservato.

Ogni entry contiene i seguenti campi :
`entry.isIntersecting` → true (visibile) / false (non visibile)
`entry.intersectionRatio` → % visibilità
`entry.target` → puntatore all'elemento osservato

alla funzione `new IntersectionObserver` è possibile passare un secondo parametro di configuraz

```
const observer = new IntersectionObserver(callback, {
  root: null,        // viewport (default) oppure un puntatore al contenitore
  threshold: 0.5   // se la visibilità è > 50%(default) isIntersecting=true})
```

debounce

è una strategia che consente di eseguire una certa funzione (ricevuta come parametro) SOLO SE è passato un certo tempo dall'ultima chiamata.

```
function debounce(func, delay) {
    let timeout;
    return function (...args) {
        clearTimeout(timeout);
        timeout = setTimeout(() => func.apply(this, args), delay);
    };
}
```

`func` è la funzione da controllare
`delay` è il tempo di attesa tra una chiamata è l'altra

Esempio:

```
function onResize() {
  console.log("resize");
}
window.addEventListener("resize", debounce(onResize, 300));
```

il log appare solo al termine del ridimensionamento, non durante il trascinamento.